He recuperado y examinado la línea de base Akkad / Sargon-NNUE v0.6.179, incluyendo `factory.py`, `brains.py`, los adaptadores de jugadores y la fábrica de evaluadores.

La versión registra 12 players y 12 identificadores de evaluador. Sin embargo, no todas sus combinaciones son válidas.

1. Matriz de compatibilidad de players y evaluadores

Player	Evaluadores admitidos	Programa externo cuando corresponde
`random`	`null`	Ninguno
`sargon`	`classic`, `npz`, `cerebrum`, `cerebrum-1.1`, `cerebrum-2`, `stockfish`, `lc0`, `tinygyal`, `znpz`, `zclassic`, `grumpy`	—
`sargon-plus`	Los mismos 11 evaluadores de `sargon`	—
`cerebrum`	Los mismos 11 evaluadores de `sargon`	—
`doc`	Los mismos 11 evaluadores de `sargon`	—
`alphazero`	`lc0`, `tinygyal`, `znpz`, `zclassic`	—
`stockfish`	`null` exclusivamente	`stockfish.exe`
`sunfish`	No requiere evaluador; la configuración admite los no nulos	Ejecutable UCI indicado por `--engine-path`
`lc0`	`null` exclusivamente	`lc0.exe`
`external`	`null` exclusivamente	Motor UCI remoto por TCP
`picosargon`	`null` exclusivamente	Ejecutable picoSargon UCI indicado por `--engine-path`
`picoalpha`	`null` exclusivamente	`picoAlpha.exe` (INT8)

La tabla refleja las restricciones implementadas y no constituye una prueba de funcionamiento de cada combinación admitida.

2. Inventario de evaluadores

Evaluador	Característica
`null`	Sin evaluación independiente; el player decide
`classic`	Evaluación clásica Sargon
`npz`	Red neuronal Sargon-NNUE
`cerebrum`	Red Cerebrum, implementación actual

Evaluador	Característica
`cerebrum-1.1`	Red Cerebrum versión 1.1
`cerebrum-2`	Alias de la implementación actual Cerebrum
`stockfish`	Evaluación posicional mediante Stockfish
`lc0`	Evaluación neural Leela Chess Zero
`tinygyal`	Evaluación neural TinyGyal
`znpz`	Evaluador NPZ con interfaz política/valor
`zclassic`	Evaluador clásico con interfaz política/valor
`grumpy`	Red neuronal compacta Grumpy

3. Casos especiales con `evaluator=null`

Player	Ejecución	Dependencia adicional
`random`	Interna, Python	Ninguna
`stockfish`	Proceso externo `stockfish.exe`	`--engine-path`
`lc0`	Proceso externo `lc0.exe`	`--lc0-uci-path` o `--engine-path`
`picosargon`	Proceso externo UCI nativo	`--engine-path` y pesos `grumpy.bin`
`picoalpha`	Proceso externo `picoAlpha.exe` INT8	`--engine-path` y `--network`
`external`	Conexión TCP a motor UCI	`--url host:puerto`

En particular, `picosargon` no ejecuta directamente el firmware de Raspberry Pi Pico: su adaptador actual lanza una versión UCI nativa mediante un proceso en Windows.

4. Inconsistencias detectadas

La revisión muestra tres aspectos que conviene mantener identificados:

1. AlphaZero y Grumpy: `brains.py` admite `alphazero + grumpy`, pero `factory.py` lo rechaza y Grumpy no implementa `evaluate_pv()`. Por lo tanto, la combinación no es funcional con el código actual.
2. Sunfish: acepta un evaluador no nulo en la configuración, pero delega sus decisiones a un programa UCI externo. El evaluador seleccionado no participa realmente en la elección de jugadas.
3. Validación dispersa: las reglas se repiten en `brains.py`, `factory.py` y `sargon_match.py`, con algunos mensajes de error desactualizados. Esto representa un riesgo de nuevas inconsistencias.

Recomendación: conservar esta matriz como referencia de la v0.6.179 y, antes de agregar más players, unificar las reglas de compatibilidad en una única fuente de verdad. Por ahora no recomiendo modificar la implementación sin una prueba que confirme un defecto operativo concreto.